

DSA STUDY BLUEPRINT SERIES

50. Introduction to Linked List

Linked Nodes, List Insertion, and Traversals

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Structure:** Nodes connected dynamically via pointers.
- **Traversals:** Linear iteration from head node pointer up to tail.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
struct Node {  
    int val;  
    Node* next;  
    Node(int x) : val(x), next(nullptr) {}  
};  
  
void printList(Node* head) {  
    while (head != nullptr) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Insert/Delete (Ends)	$O(1)$
Search Element	$O(N)$

Key Practices & Gotchas

- **Important:** Dummy sentinel nodes simplify pointer adjustments during head insertion/deletion.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment 10 → 20:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	curr->next = prev	10 points to nullptr
2	[10, 20, nullptr]	curr->next = prev	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.